

5954001 - Introdução à Computação I (Optativa para Química)

Ponteiros e Alocação Dinâmica de Memória

Prof. Dr. Denis M. L. Martins

DCM | FFCLRP | USP



Objetivos de Aprendizagem

1. Explicar **endereço, ponteiro, referência e tempo de vida** de um objeto em C.
2. Usar corretamente os operadores **&** e *****, inclusive para **leitura e escrita indireta**.
3. Relacionar **arrays, ponteiros e aritmética de endereços** sem acessar memória fora dos limites.
4. Usar ponteiros em **funções**, ponteiros para ponteiros (**T ****) e **const**.
5. Gerenciar memória dinâmica com **malloc**, **calloc**, **realloc** e **free**.

Roteiro da Aula

Fundamentos

- Modelo de memória
- Ponteiros, `&` e `*`
- Ponteiros em funções
- Arrays e aritmética
- `const`, `void *` e `T **`

Memória dinâmica

- Duração automática, estática e dinâmica
- `malloc`, `calloc`, `realloc`, `free`
- Tempo de vida e propriedade
- Erros clássicos e ferramentas
- Matrizes dinâmicas

Motivação: trocar dois valores

Comece com um programa que:

- leia dois inteiros **x** e **y**;
- troque os valores usando uma variável temporária;
- imprima os valores após a troca.

```
int temp = x;  
x = y;  
y = temp;
```

Em seguida faremos a mesma operação dentro de uma **função**. Para alterar as variáveis do chamador, precisaremos de seus **endereços**.

Ao longo da aula, use o [C Tutor](#) para visualizar a memória.

Relembrando: Modelo de Memória

A memória pode ser modelada como uma sequência de **bytes endereçáveis**.

- Cada objeto ocupa uma região de memória.
- O endereço identifica **onde** o objeto está.
- O tipo informa ao compilador **como interpretar** os bytes naquele endereço.

```
int x = 42;  
  
printf("x = %d\n", x);  
printf("&x = %p\n", (void *)&x);  
printf("sizeof x = %zu bytes\n", sizeof x);
```

%p

deve receber um argumento do tipo

void *

O que é um Ponteiro?

Um ponteiro é uma variável cujo **valor é um endereço de memória**.

```
int idade = 25;  
int *p_idade = &idade;
```



- **p_idade** guarda **&idade**.
- ***p_idade** acessa o conteúdo de **idade**.

Declaração de Ponteiros: atenção à sintaxe

```
int *p;  
float *q;  
char *s;
```

O tipo apontado determina como o endereço será interpretado.

```
int *p, q;
```

Neste caso:

- **p** é **int ***;
- **q** é **int**.

Uma convenção legível é declarar um ponteiro por linha quando isso reduzir ambiguidades.

Operador de Endereço &

O operador **&** produz o **endereço de um objeto**.

```
int x = 10;  
int *p = &x;  
  
printf("x  = %d\n", x);  
printf("&x = %p\n", (void *)&x);  
printf("p  = %p\n", (void *)p);
```

Neste exemplo:

```
p == &x
```

&x responde à pergunta: **“onde x está armazenado?”**

Operador de Desreferência *

Quando **p** aponta para um objeto válido, ***p** designa o **objeto apontado**.

```
int x = 10;
int *p = &x;

printf("%d\n", *p); // leitura indireta: 10

*p = 25;           // escrita indireta
printf("%d\n", x); // 25
```

***** permite tanto **ler** quanto **alterar** o objeto apontado.

Desreferenciar um ponteiro que não aponta para um objeto válido causa **comportamento indefinido**.

`p`, `&p`, `*p` e `&x` são coisas diferentes

```
int x = 42;  
int *p = &x;
```

Expressão	Significado
<code>x</code>	valor de <code>x</code>
<code>&x</code>	endereço de <code>x</code>
<code>p</code>	endereço armazenado em <code>p</code> – aqui, <code>&x</code>
<code>*p</code>	objeto apontado por <code>p</code> – aqui, <code>x</code>
<code>&p</code>	endereço da própria variável <code>p</code>

`p == &x` e `*p == x`

Exercício: Rastreamento de Endereços

Analise sem compilar:

```
int A = 5;  
int B = 10;  
int *pA = &A;  
int *pB = &B;
```

Expressão	Resultado
pA	?
*pA	?
&B	?
pB == &B	?
*pA + *pB	?

Desafio: Rastreamento com Mudança de Apontamento

```
int a = 10;  
int b = 20;
```

```
int *p = &a;  
int *q = &b;
```

```
*p = *q;  
q = p;  
*q = 35;
```

Determine:

1. valores finais de **a** e **b**;
2. para onde **p** e **q** apontam;
3. resultados de **p == q** e ***p == *q**.

Ponteiros em Funções: C passa argumentos por valor

Em C, os argumentos de uma função são **copiados** para seus parâmetros.

```
void tenta_alterar(int x) {  
    x = 99;  
}
```

```
int a = 10;  
tenta_alterar(a);  
printf("%d\n", a); // 10
```

Para modificar **a**, a função precisa receber **seu endereço**.

Ponteiros em Funções: alterando o objeto do chamador

```
void altera(int *p) {  
    *p = 99;  
}  
  
int main(void) {  
    int a = 10;  
  
    altera(&a);  
    printf("%d\n", a); // 99  
}
```

Fluxo conceitual:

```
main: a ---- endereço ----> parâmetro p  
                             |  
                             +----- *p altera a
```

Aplicação: **swap** com Ponteiros

```
void troca(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Uso:

```
int x = 10;
int y = 20;

troca(&x, &y);
```

Os endereços são passados **por valor**, mas permitem acesso indireto aos objetos originais.

const e Ponteiros em Parâmetros

Use **const** quando a função deve **ler**, mas não modificar, os dados apontados.

```
void imprime(const int *v, size_t n) {  
    for (size_t i = 0; i < n; ++i)  
        printf("%d ", v[i]);  
}
```

- **const int *p**: não altere o **int** através de **p**.
- **int * const p**: o próprio ponteiro não pode mudar de endereço.

Em APIs, **const** funciona como parte do **contrato da função**.

Arrays e Ponteiros: relação fundamental

```
int v[5] = {10, 20, 30, 40, 50};  
int *p = v;
```

Na maioria das expressões, **v** é convertido para um ponteiro para **v[0]**.

```
p == &v[0]  
v[2] == *(v + 2)  
p[2] == *(p + 2)
```

```
v / &v[0]  
  |  
  v  
[10][20][30][40][50]  
 ^      ^  
 p      p+2
```

Decaimento de Array

```
int v[5];
```

Em muitas expressões, **v** é convertido para **int *** apontando para **v[0]**.

Mas há exceções importantes:

```
sizeof v    // tamanho do array inteiro  
&v         // ponteiro para o array inteiro: int (*)[5]
```

Compare:

```
int *p = v;  
printf("%zu\n", sizeof v); // 5 * sizeof(int)  
printf("%zu\n", sizeof p); // tamanho de um ponteiro
```

Arrays como Parâmetros de Funções

Estas declarações de parâmetro são ajustadas para `int *`:

```
void f(int v[]);  
void f(int v[10]);  
void f(int *v);
```

Por isso a função **não descobre o tamanho do array** com `sizeof(v)`.

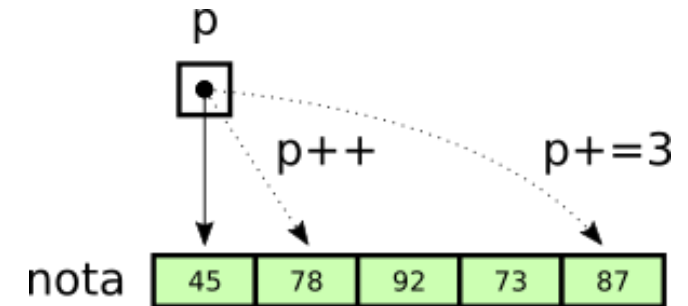
Padrão recomendado:

```
void processa(int *v, size_t n);
```

■ Passe explicitamente a quantidade de elementos, ou use outra convenção de término.

Aritmética de Ponteiros

- Ponteiros permitem navegar por estruturas de dados lineares (arrays, listas).
- O incremento é **escalado** pelo tamanho do tipo apontado: **p + 1** avança **sizeof(*p)** bytes.
- Se **p** aponta para um elemento de um array, **p + 1** aponta para o **próximo elemento**.



Fonte da imagem: [Prof. Maziero](#).

```
int nota[5] = { 45, 78, 92, 73, 87 };
int *p = nota; // p aponta para nota[0]

printf("%d\n", *p); // 45
p++; // Avança 4 bytes (em arquitetura típica)
printf("%d\n", *p); // 78
p += 3; // Pula para nota[4]
printf("%d\n", *p); // 87
```

Limites da Aritmética: não “ande pela RAM”

A aritmética de ponteiros deve permanecer associada ao **mesmo array**.

```
[10] [20] [30] [40] [50]
  ^           ^     ^
  v         v+4 v+5
              one-past
```

- É válido **formar** **$v + 5$** .
- Não é válido desreferenciar **$*(v + 5)$** .
- Acessar fora dos limites produz comportamento indefinido.

one-past-the-end é útil como sentinela de término de percurso.

Exemplo: percorrendo um intervalo

```
int soma(const int *inicio, const int *fim) {  
    int total = 0;  
  
    while (inicio < fim) {  
        total += *inicio;  
        ++inicio;  
    }  
  
    return total;  
}
```

Uso:

```
int v[] = {10, 20, 30, 40, 50};  
int total = soma(v, v + 5);
```

Este padrão combina **array decay**, **const**, **aritmética** e **one-past**.

Diferença entre Ponteiros

Dentro do mesmo array, a subtração informa a distância em **elementos**.

```
int v[10];  
int *a = &v[2];  
int *b = &v[8];  
  
ptrdiff_t d = b - a; // 6 elementos
```

Inclua:

```
#include <stddef.h>
```

Comparar ou subtrair ponteiros arbitrários, sem relação com o mesmo array, não é um mecanismo geral para medir endereços da memória.

Exercício: Inversão Genérica usando Dois Ponteiros

Implemente:

```
void inverta(int *v, size_t n);
```

Regras:

- não use `v[i]`;
- use um ponteiro para o início e outro para o fim;
- funcione para qualquer `n`, inclusive `0` e `1`.

```
esq -> [10][20][30][40][50] <- dir
```

Strings como Aplicação de Ponteiros

Uma string C termina com o caractere nulo `\0`.

```
size_t comprimento(const char *s)
{
    const char *inicio = s;

    while (*s != '\0')
        ++s;

    return (size_t)(s - inicio);
}
```

O ponteiro percorre a sequência até a sentinela.

Exercício de extensão: implementar uma cópia de string usando apenas ponteiros.

Ponteiro para Array: `int (*p)[5]`

As três declarações abaixo são diferentes:

```
int *p1;           // ponteiro para int
int a[5];          // array de 5 int
int (*p2)[5];      // ponteiro para array de 5 int
```

Exemplo:

```
int linha[5] = {1, 2, 3, 4, 5};
int (*p)[5] = &linha;

printf("%d\n", (*p)[2]); // 3
```

Os parênteses são essenciais: `int *p[5]` seria um **array de 5 ponteiros**.

Ponteiros para Ponteiros: **T ****

Um ponteiro também é um objeto e, portanto, possui endereço.

```
int x = 10;
int *p = &x;
int **pp = &p;

printf("%d\n", x);      // 10
printf("%d\n", *p);     // 10
printf("%d\n", **pp);   // 10
```

pp -----> p -----> x
 endereço 10

****pp = 50;** altera **x** para **50**.

T** em Funções: alterar o próprio ponteiro

Se uma função precisa modificar o ponteiro do chamador, ela recebe **o endereço do ponteiro**.

```
int aloca_vetor(int **v, size_t n)
{
    *v = malloc(n * sizeof **v);
    return *v != NULL;
}
```

Uso:

```
int *dados = NULL;

if (aloca_vetor(&dados, 100)) {
    /* usar dados */
    free(dados);
}
```

Aqui, **v** aponta para a variável **dados**.

Ponteiro Genérico **void ***

void * representa um ponteiro genérico para **objetos**.

```
void *mem = malloc(100);  
int *pi = mem;      // conversão implícita válida em C  
char *pc = mem;     // também válida em C
```

Em C, não é necessário fazer cast do retorno de **malloc**:

```
int *v = malloc(10 * sizeof *v);
```

O cast é obrigatório em C++, mas esta disciplina está programando em **C**.

Antes de desreferenciar, o programa precisa conhecer o tipo correto do objeto armazenado.

Três Estados Perigosos de um Ponteiro

Situação	Exemplo	Problema
Não inicializado	<code>int *p;</code>	valor indeterminado
Nulo	<code>int *p = NULL;</code>	ausência de objeto válido
Pendente (<i>dangling</i>)	aponta para objeto que deixou de existir	endereço sem objeto válido

Nunca faça:

```
*p = 10;
```

sem saber que **p** aponta para um objeto válido e vivo.

NULL: ausência explícita de um ponteiro válido

```
int *p = NULL;
```

- **NULL** é útil para representar “nenhum objeto”.
- Pode ser testado antes do uso.
- **Não** pode ser desreferenciado.

```
if (p != NULL)  
    printf("%d\n", *p);
```

NULL é um estado controlado; um ponteiro local não inicializado contém um valor indeterminado e é muito mais perigoso.

Dangling Pointer: retorno de endereço local

```
int *cria_valor(void)
{
    int x = 42;
    return &x; // ERRO: x deixa de existir ao retornar
}
```

Depois do retorno, o objeto **x** já não existe.

```
int *p = cria_valor();
printf("%d\n", *p); // comportamento indefinido
```

O endereço pode até parecer “plausível”, mas o **tempo de vida do objeto terminou**.

Dangling Pointer: depois de **free**

```
int *p = malloc(sizeof *p);  
  
if (p != NULL) {  
    *p = 42;  
    free(p);  
    p = NULL;  
}
```

Depois de **free**, o bloco deixa de pertencer ao programa.

```
free(p);  
printf("%d\n", *p); // use-after-free: comportamento indefinido
```

Atribuir **NULL** ajuda a evitar reutilização acidental daquele ponteiro específico.

Tempo de Vida e “Propriedade” da Memória

Um bom modelo mental:

ALOCAR → VALIDAR → USAR → LIBERAR → INVALIDAR REFERÊNCIAS

Perguntas que devem acompanhar cada ponteiro dinâmico:

- Quem é responsável por liberar o bloco?
- Até quando o objeto continua vivo?
- Existem outros ponteiros referenciando o mesmo bloco?
- O ponteiro ainda é válido neste ponto do código?

Em C, essas regras são convenções do programa: o compilador não faz o gerenciamento por você.

Formas de Alocação e Duração de Armazenamento

Automática

```
void f(void) {  
    int x;  
    int v[10];  
}
```

Normalmente associada à **stack**. Termina ao sair do bloco.

Estática

```
int global;  
static int contador;
```

Existe durante toda a execução do programa.

Dinâmica

```
int *v = malloc(n * sizeof *v);
```

Normalmente associada ao **heap**.

- tamanho definido em execução;
- tempo de vida controlado explicitamente;
- exige estratégia de liberação.

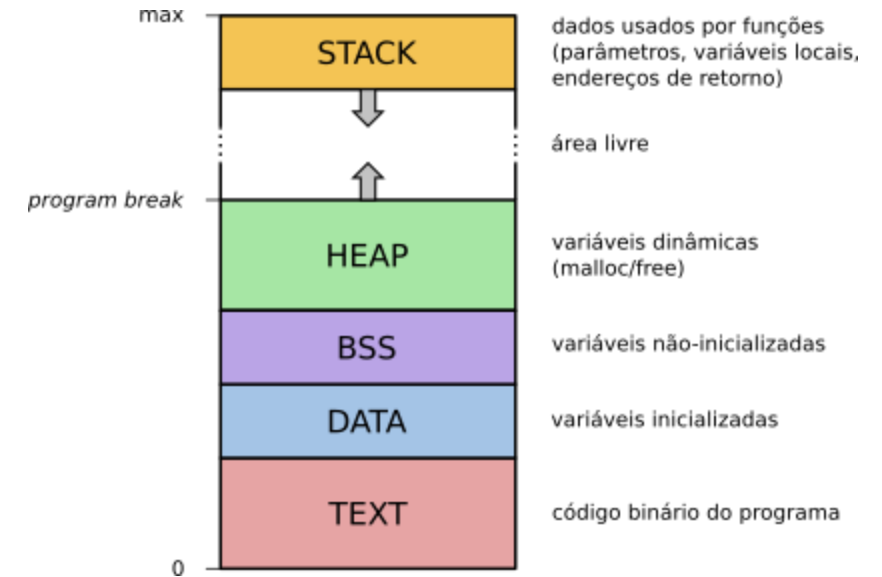
Stack e Heap no Processo

Stack

- chamadas de funções;
- parâmetros e variáveis automáticas;
- liberação automática ao retornar.

Heap

- blocos obtidos dinamicamente;
- úteis para tamanhos definidos em execução;
- permanecem vivos até serem liberados ou o processo terminar.



Fonte da imagem: [Prof. Carlos Maziero](#).

“Stack” e “heap” são modelos de implementação usuais; o conceito da linguagem é o **tempo de vida** dos objetos.

Por que usar Memória Dinâmica?

Use alocação dinâmica quando o tamanho ou o tempo de vida não é conhecido adequadamente na compilação.

Exemplos:

- quantidade de dados definida pelo usuário;
- vetores e matrizes grandes ou redimensionáveis;
- estruturas como listas, árvores e grafos;
- objetos que precisam sobreviver à função que os criou.

```
size_t n;  
scanf("%zu", &n);  
  
int *v = malloc(n * sizeof *v);
```

malloc: alocação de bytes

```
#include <stdlib.h>

int *v = malloc(100 * sizeof *v);

if (v == NULL) {
    /* tratar falha */
    return 1;
}
```

- **malloc** retorna o endereço do bloco ou **NULL**. O conteúdo inicial dos bytes é **indeterminado**.
- É seguro **escrever** no bloco válido. Não é seguro **ler um objeto não inicializado**.

```
v[0] = 5;           // válido: inicializa o elemento
printf("%d", v[1]); // errado se v[1] ainda não foi inicializado
```

Padrão Recomendado: `sizeof *ponteiro`

Prefira:

```
int *v = malloc(n * sizeof *v);
```

a:

```
int *v = malloc(n * sizeof(int));
```

Vantagem: o tamanho acompanha automaticamente o **tipo do ponteiro** se ele mudar.

```
double *v = malloc(n * sizeof *v);
```

Para tamanhos vindos de entrada externa (por exemplo, lidos do teclado), programas robustos também validam possíveis **overflows** no cálculo `n * sizeof *v`.

calloc: alocar e zerar os bytes

```
int *v = calloc(n, sizeof *v);
```

calloc recebe:

1. quantidade de elementos;
2. tamanho de cada elemento.

O bloco retornado tem seus **bytes inicializados com zero**.

```
if (v == NULL) {  
    /* tratar falha */  
}
```

Não confunda **calloc** com “construir objetos”: em C ele apenas reserva armazenamento e zera bytes.

realloc: redimensionando um bloco

```
int *tmp = realloc(v, novo_n * sizeof *v);

if (tmp != NULL) {
    v = tmp;
    n = novo_n;
} else {
    /* v continua apontando para o bloco antigo */
}
```

Por que usar um ponteiro temporário?

```
v = realloc(v, novo_n * sizeof *v); // arriscado
```

Se a operação falhar, **realloc** retorna **NULL**; sobrescrever **v** pode fazer o programa **perder a única referência ao bloco antigo**.

free: encerrando o tempo de vida do bloco

```
int *p = malloc(sizeof *p);

if (p != NULL) {
    *p = 10;
    /* ... uso ... */
    free(p);
    p = NULL;
}
```

Regras fundamentais:

- libere somente endereços obtidos por funções de alocação compatíveis;
- cada bloco deve ser liberado **no máximo uma vez**;
- **free(NULL)** é permitido e não faz nada;
- não use o bloco após **free**.

Memory Leak: perdendo a referência ao bloco

```
int *p = malloc(100 * sizeof *p);  
if (p == NULL) return 1;  
  
/* ... */  
  
p = malloc(200 * sizeof *p); // PROBLEMA
```

antes: p -----> [bloco A]

depois: [bloco A] <- continua alocado,
 p -----> [bloco B] mas seu endereço foi perdido

O bloco A só será recuperado quando o processo terminar.

Em programas longos ou repetitivos, leaks acumulados podem consumir grandes quantidades de memória.

Erros Clássicos de Memória Dinâmica

Erro	Exemplo	Consequência
Leak	perder a última referência	memória permanece alocada
Use-after-free	usar <code>*p</code> após <code>free(p)</code>	comportamento indefinido
Double free	liberar o mesmo bloco duas vezes	comportamento indefinido
Out-of-bounds	acessar além do bloco	comportamento indefinido
Leitura não inicializada	ler conteúdo antes de gravar	valor indeterminado / UB em vários casos

Muitos bugs de memória parecem funcionar em um teste e falhar em outro. “**Não travou**” não significa “**está correto**”.

Ferramentas para Encontrar Erros

Compile com avisos rigorosos:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -g programa.c -o programa
```

AddressSanitizer + UndefinedBehaviorSanitizer:

```
gcc -std=c17 -g -fsanitize=address,undefined programa.c -o programa  
./programa
```

Em ambientes com Valgrind:

```
valgrind --leak-check=full ./programa
```

Use ferramentas **durante o desenvolvimento**, não apenas quando o programa já falhou.

Matrizes Dinâmicas: o Problema

Para uma matriz **linhas × colunas**, precisamos decidir:

- como os elementos serão organizados na memória;
- como calcular **mat[i][j]**;
- quantas alocações serão feitas;
- como liberar corretamente todos os blocos.

Três estratégias úteis:

1. um único bloco linearizado;
2. vetor de ponteiros + uma alocação por linha;
3. vetor de ponteiros + bloco contíguo.

Matriz Dinâmica: bloco único

```
size_t L = 4, C = 6;  
int *mat = malloc(L * C * sizeof *mat);  
  
if (mat == NULL)  
    return 1;
```

Acesso:

```
mat[i * C + j] = 0;
```

Liberação:

```
free(mat);
```

Vantagens: uma alocação, memória contígua, boa localidade.

Custo: o índice bidimensional precisa ser linearizado explicitamente.

Matriz Dinâmica: `int **`

Primeiro alocamos o vetor de ponteiros para as linhas:

```
int **mat = malloc(L * sizeof *mat);  
if (mat == NULL)  
    return 1;
```

Depois, cada linha:

```
for (size_t i = 0; i < L; ++i) {  
    mat[i] = malloc(C * sizeof *mat[i]);  
    if (mat[i] == NULL) {  
        /* liberar linhas já alocadas */  
    }  
}
```

Acesso natural: `mat[i][j] = 0;`

Matriz com `int **`: liberação correta

Se todas as linhas foram alocadas:

```
for (size_t i = 0; i < L; ++i)
    free(mat[i]);

free(mat);
mat = NULL;
```

A ordem importa conceitualmente:

```
mat -> [p0][p1][p2]...
        |  |  |
        v  v  v
    linha linha linha
```

Libere primeiro os blocos apontados pelas linhas; depois o vetor de ponteiros.

Matriz Dinâmica: acesso **mat[i][j]** + bloco contíguo

```
int **mat = malloc(L * sizeof *mat);
int *dados = malloc(L * C * sizeof *dados);

if (mat == NULL || dados == NULL) {
    free(mat);
    free(dados);
    return 1;
}

for (size_t i = 0; i < L; ++i)
    mat[i] = dados + i * C;
```

Agora: **mat[i][j] = 0;**

Liberação:

```
free(dados);
free(mat);
```

Comparando Estratégias de Matriz

Estratégia	Alocações	Contígua?	Acesso
bloco linear	1	sim	$m[i * C + j]$
<code>int **</code> por linha	$L + 1$	não necessariamente	$m[i][j]$
ponteiros + bloco	2	sim	$m[i][j]$

Para uma primeira implementação:

- use **bloco linear** quando simplicidade de armazenamento e desempenho importarem;
- use **linhas separadas** quando tamanhos de linhas puderem variar;
- use **ponteiros + bloco contíguo** quando quiser $m[i][j]$ e armazenamento linear.

Exercício: Matriz Dinâmica

Escreva um programa que:

1. leia **L** e **C**;
2. aloque uma matriz dinâmica de **double** em **bloco único**;
3. preencha **m[i*C+j]** com **i + j / 10.0**;
4. calcule a média dos elementos;
5. libere toda a memória antes de encerrar.

Desafio: reescreva usando **double **** e compare a estratégia de liberação.

Exercício de Diagnóstico: encontre todos os erros

```
int *v = malloc(5 * sizeof *v);  
  
for (int i = 0; i <= 5; ++i)  
    printf("%d\n", v[i]);  
  
free(v);  
v[0] = 10;  
free(v);
```

Identifique e classifique cada problema.

Boas Práticas: Checklist

Antes de desreferenciar um ponteiro, pergunte:

- ele foi inicializado?
- aponta para um objeto vivo e do tipo esperado?
- o acesso está dentro dos limites do array/bloco?

Para memória dinâmica:

- teste o retorno da alocação;
- use `sizeof *p`;
- defina claramente quem libera;
- nunca use um bloco depois de `free`.

Conclusão

- Ponteiros armazenam **endereços**, e ***p** acessa o objeto apontado.
- C passa argumentos por valor; ponteiros permitem **alteração indireta**.
- Arrays e ponteiros são relacionados, mas **não são a mesma coisa**.
- Aritmética é válida dentro dos limites do mesmo array, incluindo a sentinela *one-past*.
- **void *** é genérico e **não exige cast do malloc** em C.
- **malloc**, **calloc**, **realloc** e **free** controlam o tempo de vida de objetos dinâmicos.
- Leaks, dangling pointers e acessos inválidos são bugs de **tempo de vida e limites**.
- Matrizes dinâmicas exigem escolher conscientemente o **layout de memória**.

Material Adicional

- Ponteiros — Prof. Carlos Maziero
- Alocação de Memória — Prof. Carlos Maziero
- Alocação Dinâmica de Matrizes — Prof. Carlos Maziero
- C Tutor — visualização de execução
- Snefru: Learning Programming with C

Exercícios e Questões

Questões Teórica: Fundamentos

1. Considere:

```
int a = 5;  
int *p = &a;  
*p = 10;
```

O valor final de **a** é: (A) 5 (B) 10 (C) endereço de **a** (D) indefinido

2. Explique a diferença entre:

```
int *p;  
int p[5];  
int (*p)[5];
```

3. Por que **p == NULL** pode ser testado, mas ***p** não pode ser usado nesse estado?

Questões Teóricas: Arrays e Funções

4. Explique por que `sizeof(v)` dentro de `void f(int v[])` não retorna o tamanho do array original.

5. Qual a diferença entre alterar `*p` e alterar `p`?

6. Analise:

```
int v[4] = {10, 20, 30, 40};  
int *p = v + 4;
```

É válido criar `p`? É válido executar `printf("%d", *p);`?

Questões Teóricas: Memória Dinâmica

7. Qual a diferença conceitual entre **malloc** e **calloc**?

8. Por que o padrão abaixo é mais seguro?

```
int *tmp = realloc(v, novo_n * sizeof *v);  
if (tmp != NULL)  
    v = tmp;
```

9. Classifique os bugs:

- perder a última referência a um bloco ainda alocado;
- usar o bloco depois de **free**;
- chamar **free** duas vezes no mesmo bloco.

Exercício Prático 1: Estatísticas por Ponteiros

Implemente:

```
void estatisticas(const double *v, size_t n,  
                 double *min, double *max, double *media);
```

Requisitos:

- percorra o vetor usando ponteiros;
- retorne os três resultados por parâmetros de saída;
- trate `n == 0` de forma documentada.

Exercício Prático 2: Vetor Dinâmico Redimensionável

Escreva um programa que:

1. leia uma quantidade inicial **n**;
2. aloque um vetor de **int**;
3. preencha os elementos;
4. leia um novo tamanho **m**;
5. use **realloc** com ponteiro temporário;
6. inicialize os novos elementos, caso **m > n**;
7. imprima o vetor e libere a memória.

Exercício Prático 3: Matriz e Maior Elemento

Implemente uma matriz $L \times C$ de inteiros usando **um único bloco contíguo**.

O programa deve:

- ler dimensões e elementos;
- imprimir a matriz;
- encontrar valor máximo e suas coordenadas;
- liberar a memória corretamente.

Restrição: acesse os elementos com:

```
m[i * C + j]
```

Exercício Prático 4: Caça aos Bugs com Sanitizer

Compile o programa abaixo com AddressSanitizer e corrija todos os problemas:

```
int *p = malloc(3 * sizeof *p);  
p[0] = 10;  
p[1] = 20;  
p[3] = 40;  
free(p);  
printf("%d\n", p[0]);
```

Comando sugerido:

```
gcc -std=c17 -g -Wall -Wextra -fsanitize=address,undefined bug.c -o bug
```

Desafio Final: API de Matriz Dinâmica

Projete as funções:

```
double *mat_cria(size_t L, size_t C);  
void mat_libera(double **m);  
double mat_get(const double *m, size_t C, size_t i, size_t j);  
void mat_set(double *m, size_t C, size_t i, size_t j, double valor);
```

Objetivos:

- encapsular cálculo de índices;
- usar **const** onde adequado;
- definir claramente **quem possui e libera** a memória;
- documentar pré-condições de índices e dimensões.

Dúvidas e Discussão